NPLAN-4241 NS Client FIPS Mode Support Test Plan

⚠ The document is under construction.

- Summary
  - Feature Description
  - Team
  - Checklist
- Requirements
  - Hardware & Environment Requirements
  - Control Flags and Feature Flags
- Customer Use Cases
- Test Plan
  - Goal
  - Scope
    - Out of Scope
    - Status
  - Test Automation
- Cross-functional Coordination
  - Integration Testing
  - Regression Testing
  - E2E Performance Testing
  - E2E Functional / Customer Scenario Testing
- Schedule
  - Project Milestones
  - Release Schedule
- Reporting
- Related Documents

Summary

Name of the Feature	NS Client FIPS Mode Support
NPLAN & Epic	• NPLAN-4241: Support FIPS-140-3 compliance requirements IN PROGRESS • NPLAN-4147: FIPS 140-3 support for NPA Client BACKLOG
Product Requirement Document	

Design Document	NS Client FIPS Mode Support
Planned Release	Beta: Apr 2026 / R137 GA: June 2026 / R139
TestRail Link	TBD

Feature Description

This feature introduces FIPS 140-3 compliant cryptographic operation for Netskope Client (NSC) on Windows and macOS using the OpenSSL 3.x provider architecture.

Key aspects:

- FIPS Mode is controlled by an integer flag `FipsMode`:
  - `0` – Non-FIPS Mode
  - `1` – Strict FIPS Mode (fail-close if FIPS init fails)
  - `2` – Permissive FIPS Mode (prefer FIPS; fallback to Non-FIPS if init fails)
- Platform scope (Phase 1): Windows (64-bit, including server) and macOS.
- FIPS Mode covers NS Agent, NS Service, and TLS-using modules (e.g. tlspipe, nssl, nsHttpClient, AppProbStatsComm).
- FIPS provider (OpenSSL 3.1.2 FIPS 140-3) is loaded at runtime while core libraries use OpenSSL 3.5.2.
- TLS cipher suites are restricted to FIPS-approved suites when FIPS Mode is active.
- Installation/uninstallation manages FIPS provider binaries and configuration (`fips.dll` / `fips.dylib`, `fipsmodule.cnf`, `openssl` subdirectory).
- Logging is enhanced with `FIPS:`-prefixed entries in `nsdebuglog.log` and `nsAppUI.log`.

Team

QE Lead / Manager / Director	@Rajesh Narayan Bhat
Other QEs	@Anand Kumar S @Austin Cheng
Dev Lead / Manager / Director	@Baoyue Hu @Frank Hu

Service Owner	
Product Manager(s) (Mandatory)	

Checklist

Following checklist contains common questions to consider in the test planning:

Applicable	
<Yes>	PRD, Design documents are reviewed and signed off by all stack holder
<Yes>	PRD, Design documents are reviewed and signed off by QE

Requirements

Hardware & Environment Requirements

No specific hardware & environment requirements are needed

Refer to Release Stack Reference for an up-to-date list of all shared stacks used in release testing.

Test Phase	**Environment**	**Changes Needed**
Component Testing	NPA01 (with feature build) or QA01 (with develop build)	
Performance Testing	NPA01 (with feature build) or QA01 (with develop build)	
Integration Testing	QA01 (with develop build)	
Regression Testing	STG01 and STG01-mplegacy (with release build)	
E2E Performance Testing	TBD	

Customer Use Cases

- As an IT admin in a regulated environment, I can configure NS Client to operate in Strict FIPS Mode so that all cryptographic operations are performed using validated FIPS 140-3 modules and cipher suites.
- As an IT admin who prefers availability over strict enforcement, I can configure Permissive FIPS Mode so that NS Client attempts to use FIPS-validated crypto but falls back to Non-FIPS Mode if initialization fails, with clear logging.
- As a security/compliance owner, I can verify through logs and configuration that NS Client is running in FIPS Mode and only using FIPS-compliant cipher suites for TLS 1.2 and TLS 1.3 connections.
- As an operator, I can install, upgrade, and uninstall NS Client with FIPS Mode enabled without leaving stale FIPS provider binaries or registry/preferences entries behind.

Test Plan

Goal

- Validate that NS Client correctly enables and enforces FIPS Mode (Strict and Permissive) on Windows and macOS for NS Agent and NS Service
- Ensure FIPS provider loading, self-tests, and runtime conditional tests behave as designed.
- Confirm TLS-using modules (nsHttpClient, nssl, tlspipe, AppProbStatsComm) only negotiate FIPS-approved cipher suites in FIPS Mode.
- Verify error handling and fallback behaviors:
  - Strict FIPS Mode causes fail-close if FIPS init/self-test fails.
  - Permissive FIPS Mode falls back to Non-FIPS Mode with clear logging.
- Ensure proper install/upgrade/uninstall handling of FIPS provider binaries and configuration.
- Confirm there is no regression in Non-FIPS Mode behavior or performance.

Scope

The testing covers all relevant scenarios for:

- Windows (x64 desktop + server) and macOS (Intel + Apple Silicon where applicable).
- FipsMode values: 0 (Non-FIPS), 1 (Strict), 2 (Permissive).

- NS Agent + NS Service + TLS-using modules.
- Installer/upgrade/uninstall flows that impact FIPS binaries and configuration.

Out of Scope

- Linux, iOS, Android FIPS certification behavior (Phase 2 / future work).
- Non-OpenSSL crypto providers or OS-native FIPS modes outside NS Client.
- UI localization beyond basic verification of FIPS-related messages where applicable.

Status

Test completion: **0% (0/30)**

Test Status Example: PASS FAIL NA BLOCK

Test Types	ID	Priority	Test Description	Platform	Test Status	Comments
Functional	1	P0	**FipsMode=0 Baseline – Non-FIPS Mode:** Install NS Client with `FipsMode=0` (registry/preferences and installer). Verify client operates as current baseline: FIPS provider not loaded, no FIPS self-test logs, existing cipher suite configuration preserved.			
Functional	2	P0	**Strict FIPS Mode – Initial Enable:** Install client with `FipsMode=1`. On first startup of NS Agent and NS Service, verify FIPS provider is loaded, POSTs run, and logs show `FIPS: Enabled successfully` with Approved mode status.	Win x64 / Server x64 / macOS		
Functional	3	P0	**Permissive FIPS Mode – Successful FIPS Init:** Install with `FipsMode=2` and valid FIPS provider. Verify client behaves identically to Strict FIPS Mode when initialization succeeds; logs show FIPS mode active but indicate Permissive policy.	Win x64 / Server x64 / macOS		

Functional	4	P0	**Strict FIPS Mode – Failure Path:** Corrupt or remove FIPS provider binary/config, then start client with `FipsMode=1`. Verify provider load/self-test failure is logged and NS Client enters disabled/fail-close state (no TLS tunnels, no policy enforcement), with explicit error logs.	Win x64 / macOS		
Functional	5	P0	**Permissive FIPS Mode – Failure Path:** Corrupt/remove FIPS provider, start client with `FipsMode=2`. Verify failure is logged and client automatically falls back to Non-FIPS Mode (normal functionality), with clear indication of compliance downgrade.	Win x64 / macOS		
Functional	6	P0	**NS Agent + Service Both in FIPS Mode:** Confirm that both NS Agent and NS Service separately enable FIPS Mode and that logs from each process show successful FIPS initialization. If one fails in Strict Mode, verify overall client is disabled as per design.	Win x64 / Server x64 / macOS		
Functional	7	P0	**Runtime Switch – Non-FIPS → Strict FIPS:** Start in `FipsMode=0`, then change registry/preferences to `1` and restart client. Verify FIPS init runs and client moves into Strict FIPS Mode correctly.	Win x64 / macOS		
Functional	8	P0	**Runtime Switch – Strict → Non-FIPS:** Start in `FipsMode=1`, then change to `0` and restart client. Verify FIPS provider is not loaded and baseline cipher suites are restored.	Win x64 / macOS		
Functional	9	P1	**Runtime Switch – Strict → Permissive:** Change `1 → 2` and restart. Verify behavior changes only in failure handling path (on forced provider failure) but Approved operation remains identical on success.	Win x64 / macOS		
			TLS / Cipher Suite Enforcement			

Functional	10	P0	**TLS 1.2 – FIPS Suites Only in Strict Mode:** With `FipsMode=1`, force TLS 1.2 connections (e.g., via gateway config) for each module (nsHttpClient, nssl, tlspipe, AppProbStatsComm). Capture handshake (e.g., with debug logs or network capture) and verify negotiated cipher suites are limited to documented FIPS-approved ones (e.g., `TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384`).	Win x64 / macOS		
Functional	11	P0	**TLS 1.3 – FIPS Suites Only in Strict Mode:** With `FipsMode=1`, ensure TLS 1.3 is used where supported and confirm only `TLS_AES_128_GCM_SHA256` and/or `TLS_AES_256_GCM_SHA384` are negotiated. Verify `TLS_CHACHA20_POLY1305_SHA256` is not used.	Win x64 / macOS		
Functional	12	P0	**Non-FIPS Mode Cipher Suites Unrestricted:** With `FipsMode=0`, verify that existing (possibly non-FIPS) cipher suites can still be negotiated if configured on server side, confirming no regression to legacy deployments.	Win x64 / macOS		
Functional	13	P1	**Mixed TLS Version Environments:** In environments where some services allow only TLS 1.2 and others TLS 1.3, verify NS Client in Strict Mode selects FIPS-approved suites appropriately per module/endpoint without failures.	Win x64 / macOS		
			Conditional Runtime Tests & Error Handling			
Functional	14	P0	**Key Pair Generation – Pairwise Consistency:** Trigger operations that generate RSA/ECDSA keys (e.g., certificate provisioning or ephemeral keys). Confirm no FIPS provider errors in normal conditions and verify logs remain clean.	Win x64 / macOS		
Functional	15	P1	**Injected Failure in Pairwise Test:** Using a test or debug configuration (if available) that simulates a failure in pairwise consistency tests, verify provider enters error state and subsequent crypto calls fail as	Win x64 / macOS		

			per design, with clear error logs. Client behavior should match Strict vs Permissive semantics.			
Functional	16	P0	**DRBG Continuous Test – Normal Operation:** Stress test random number generation via repeated TLS connections and key operations; confirm no unexpected provider error states or log noise.			
			Logging & Observability			
Functional	17	P0	**FIPS Log Tagging:** Verify all FIPS-related events in `nsdebuglog.log` and `nsAppUI.log` are prefixed with `FIPS:` and include mode (Strict/Permissive), provider load result, and any error states.	Win x64 / macOS		
Functional	18	P1	**Audit-Friendly Log Sequence:** From install → first startup → TLS operation → shutdown, review logs to ensure a clear sequence that can be used for audit evidence of FIPS operation.	Win x64 / macOS		
			Installation / Upgrade / Uninstallation			
Functional	19	P0	**Fresh Install – FIPS Mode Off:** Install NS Client with default `FipsMode=0`. Verify `openssl` subdirectory and FIPS binaries are present or absent per design, and no FIPS registry/preferences keys are wrongly set.	Win x64 / macOS		
Functional	20	P0	**Fresh Install – Strict FIPS Mode:** Install with `FipsMode=1`. Verify FIPS provider binaries and `fipsmodule.cnf` are installed into `openssl` subdirectory and FIPS registry/preferences key is correctly set.	Win x64 / macOS		
Functional	21	P0	**Upgrade with FipsMode=1:** Upgrade from Non-FIPS build to FIPS-enabled build with `FipsMode=1`. Confirm FIPS provider artifacts are correctly upgraded and both NS Agent and Service run in Strict FIPS Mode post-upgrade.	Win x64 / macOS		

Functional	22	P1	**Upgrade with FipsMode=2:** Similar to ID 21 but with `FipsMode=2`. Verify fallback behavior remains correct both during and after upgrade if provider load fails.	Win x64 / macOS		
Functional	23	P0	**Uninstall Cleanup:** Uninstall NS Client from Strict/Permissive FIPS Mode. Verify `openssl` subdirectory is removed and FIPS-related registry/preferences entries are cleaned up.	Win x64 / macOS		
Negative	24	P1	**Aborted Install/Upgrade:** Simulate aborted MSI/pkg install or upgrade. Verify that existing client version continues to operate correctly in its previous FipsMode and FIPS artifacts are not left in a partially upgraded state.	Win x64 / macOS		
			Negative & Compatibility Tests			
Negative	25	P0	**Unsupported Cipher – Strict Mode:** Force server side to only allow non-FIPS cipher suites and run client in `FipsMode=1`. Verify handshake fails as expected and error is logged, without falling back to non-FIPS algorithms.	Win x64 / macOS		
Negative	26	P1	**Misconfigured FipsMode Value:** Set invalid `FipsMode` (e.g., 3 or negative). Verify client behavior: either map to Non-FIPS with warning or fail safely, per implementation, and log a clear configuration error.	Win x64 / macOS		
Compatibility	27	P1	**Coexistence with Non-FIPS Clients:** In the same environment, run both FIPS and Non-FIPS NS Clients against the same backend. Verify no server-side interoperability issues and that policies apply correctly to both.	Win x64 / macOS		
			Stability / Performance			
ability	28	P1	**Long-Run FIPS Session Stability:** Run NS Client in Strict FIPS Mode for extended period (e.g., 24–72 hours) with continuous TLS usage. Verify no	Win x64 /		

			memory leaks, provider errors, or increasing FIPS log noise.	macOS		
Performance	29	P2	**FIPS vs Non-FIPS Performance:** Compare key operations and TLS throughput (e.g., tunnel establishment time, CPU usage) between Non-FIPS Mode and Strict FIPS Mode to ensure overhead is within acceptable range.	Win x64 / macOS		

Test Automation

Automate the p0 test cases.

Cross-functional Coordination

Integration Testing

- Validate integration with:
  - NPA (tlspipe).
  - SWG (nssl).
  - Backend services used by nsHttpClient.
- Confirm backend stacks are configured to support FIPS-approved cipher suites.

Regression Testing

- Include FIPS Mode permutations in core client regression runs for Windows and macOS:
  - Non-FIPS (0), Strict (1), Permissive (2).

E2E Performance Testing

No specific requirements for the Central QE / Solutions QE team

E2E Functional / Customer Scenario Testing

No specific requirements for the Central QE / Solutions QE team

Schedule

Project Milestones

TBD

Release Schedule

Beta: Apr 2026 / R137

GA: Jun 2026 / R139

Reporting

- Jenkins
- TestRail

Case ID

Related Documents